PES University – Software Engineering Lab

Lab 6: Code and Branch Coverage

PES1UG24AM393_SELAB6

Name	Roopesh P
USN	PES1UG24AM393
Section	26
Lab	Lab 6 – Code and Branch Coverage
Module Under Test	shipping_quote.py

Coverage Summary

Metric	Result
Total Tests	31
Tests Passing	31 / 31 ✓
Line Coverage	100% (53 / 53 statements)
Branch Coverage	100% (36 / 36 branches)
Tool	pytest + pytest-cov
Python Version	3.13.14

Test File: PES1UG24AM393_SELAB6.py

```
import pytest
from shipping_quote import process_shipments, monitor_container_pressures

# ■■■
```

```
def test_all_shipments_not_approved_returns_no_cargo():
    """Branch: manifest_approved False → continue (all skipped) → NO_CARGO"""
    shipments = [
        {"manifest_approved": False, "mode": "ground",
         "packages": [{"weight_kg": 10, "is_hazmat": False}]}
    ]
    result = process_shipments(shipments, 5.0)
    assert result["status"] == "NO_CARGO"

# ■■■
```

```
    }]
    result = process_shipments(shipments, 0.0)
    assert result["status"] == "NO_CARGO"

def test_heavy_package_above_30kg():
    """Branch: weight > 30 → weight * (base_rate * 1.5)"""
    shipments = [{
        "manifest_approved": True, "mode": "ground",
        "is_fragile": False, "express_priority": False,
        "packages": [{"weight_kg": 40, "is_hazmat": False}]
    }]
    result = process_shipments(shipments, 0.0)
    assert result["total_quote"] == 240.0  # 40 * (4.0 * 1.5)
    assert result["status"] == "CLEARED"

def test_package_exactly_30kg_uses_normal_rate():
    """Boundary: weight == 30 → NOT > 30 → normal rate"""
    shipments = [{
        "manifest_approved": True, "mode": "ground",
        "is_fragile": False, "express_priority": False,
        "packages": [{"weight_kg": 30, "is_hazmat": False}]
    }]
    result = process_shipments(shipments, 0.0)
    assert result["total_quote"] == 120.0  # 30 * 4.0
    assert result["status"] == "CLEARED"

# ■■■
```

```
# ■■■
```

```
        ]
    }]
    result = process_shipments(shipments, 0.0)
    assert result["total_quote"] == 890.0
    assert result["status"] == "CLEARED"

def test_shipment_with_no_packages():
    """Shipment with empty packages list → only handling_fee added"""
    shipments = [{
        "manifest_approved": True, "mode": "ground",
        "is_fragile": False, "express_priority": False, "packages": []
    }]
    result = process_shipments(shipments, 15.0)
    assert result["total_quote"] == 15.0
    assert result["status"] == "CLEARED"

# ═══════════════════════════════════════════════════════════════════════════
# monitor_container_pressures
# ═══════════════════════════════════════════════════════════════════════════

def test_empty_pressure_logs_returns_no_data():
    """Branch: not pressure_logs (empty list) → NO_DATA"""
    assert monitor_container_pressures([]) == "NO_DATA"

def test_none_pressure_logs_returns_no_data():
    """Branch: not pressure_logs (None) → NO_DATA"""
    assert monitor_container_pressures(None) == "NO_DATA"

def test_all_pressures_stable():
    """Branch: all psi >= 14.0 → streak always reset → STABLE"""
    assert monitor_container_pressures([[15.0, 16.0, 14.5, 20.0]]) == "STABLE"

def test_single_low_pressure_no_leak():
    """Branch: one psi < 14, streak = 1 (< 2); resets on next ok → STABLE"""
    assert monitor_container_pressures([[15.0, 13.0, 15.0]]) == "STABLE"

def test_two_consecutive_low_pressures_leak_detected():
    """Branch: streak >= 2 → LEAK_DETECTED"""
    assert monitor_container_pressures([[15.0, 13.0, 13.0, 15.0]]) == "LEAK_DETECTED"

def test_leak_on_first_two_readings():
    """Leak detected immediately on first two readings"""
    assert monitor_container_pressures([[13.0, 12.0]]) == "LEAK_DETECTED"

def test_leak_in_second_container():
    """LEAK_DETECTED found in second container while first is stable"""
    logs = [[15.0, 16.0], [14.5, 13.0, 12.5]]
    assert monitor_container_pressures(logs) == "LEAK_DETECTED"

def test_multiple_containers_all_stable():
    """Multiple containers, all stable → STABLE"""
    logs = [[15.0, 14.0, 16.0], [20.0, 18.0], [14.1, 14.2, 15.0]]
    assert monitor_container_pressures(logs) == "STABLE"

def test_streak_resets_after_normal_pressure():
    """Low psi then normal psi resets streak; next low doesn't trigger leak alone"""
    assert monitor_container_pressures([[13.0, 15.0, 13.0, 15.0]]) == "STABLE"

def test_exactly_at_14_psi_is_stable():
    """Boundary: psi == 14.0 is NOT < 14.0 → streak resets → STABLE"""
    assert monitor_container_pressures([[14.0, 14.0, 14.0]]) == "STABLE"

def test_just_below_14_psi_counts_toward_streak():
    """Boundary: psi == 13.999 < 14.0 → counted as low pressure → LEAK_DETECTED"""
    assert monitor_container_pressures([[13.999, 13.999]]) == "LEAK_DETECTED"
```